

BRAC UNIVERSITY
Department of Computer Science and Engineering

Set:B

Examination: Mid Term
Duration: 1 Hour 20 Minutes

Semester: Spring 2026
Full Marks: 20

CSE 420: Compiler Design

Answer the following questions. Theoretical questions should be answered in a precise, concise, and to-the-point manner. Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

1. CO3	In an LR parser for the augmented grammar $S' \rightarrow S$, why is the configuration $S' \rightarrow S.$ with lookahead \$ required for acceptance? In LR(0) item construction and SLR(1) parsing how a production rule like $A \rightarrow \epsilon$ is handled?	1.5 + 1.5
2. CO2	If a symbol table stores only the names of identifiers and omits all other associated attributes, what types of errors may arise during the compilation process?	1
3. CO3	a. Given the following grammar, produce the LR(0) automaton: 1. $S \rightarrow S S$ 2. $S \rightarrow \epsilon$ 3. $S \rightarrow digit = E ;$ 4. $E \rightarrow E boolop E$ 5. $E \rightarrow digit$ Note that the Terminals here are: $\{digit, =, ;, boolop\}$	4
	b. For the grammar above, $follow(S) = \{\$, digit\}$ $follow(E) = \{;, boolop\}$. Now, by showing the SLR Parsing Table, determine the conflicting actions (if any).	3
4. CO2	Explain how to update the lexical and syntax analyzer to be able precisely report in what line and what word number (column) an error is discovered during the semantic analysis step.	2
5. CO4	Consider Two SDT, SDT-1 and SDT-2. In the grammar rules of SDT-1 there are only semantic actions present at the end of the production rule. In SDT-2 there are semantic actions present at the beginning of the production body, in the middle of the production body and at the end of the production body and there are multiple semantic actions in a single production rule. Explain which SDT can handle more complex attribute calculation? How to implement SDT-2?	2

Please Turn Over

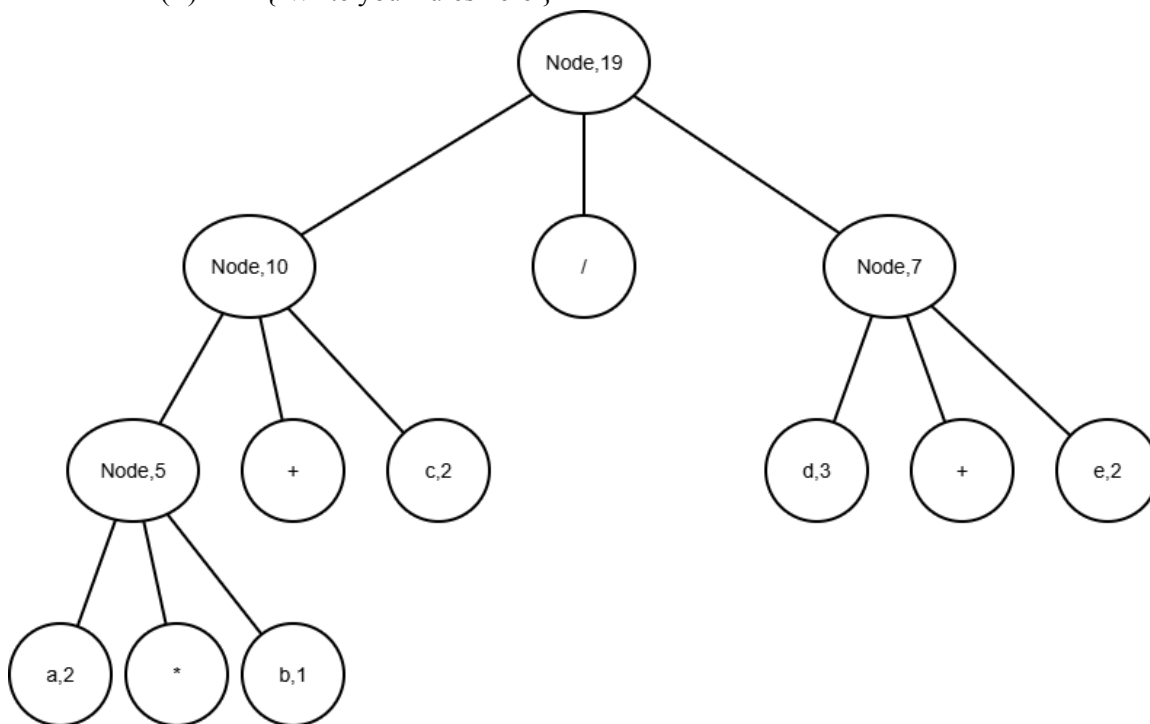
6.
CO4

Write the SDD rules for an abstract syntax tree construction for the following grammar that will store a count for the number of reduce operations being done to reach up to the current node for each node in the parse tree.

5

Input String: $(a*b+c)/(d+e)$

$E \rightarrow E_1 + T$ { Write your rules here }
 $E \rightarrow T$ { Write your rules here }
 $T \rightarrow T_1 * F$ { Write your rules here }
 $T \rightarrow T_1 / F$ { Write your rules here }
 $T \rightarrow F$ { Write your rules here }
 $F \rightarrow id$ { Write your rules here }
 $F \rightarrow (E)$ { Write your rules here }



Hunts:

For d,

the reductions are $F \rightarrow id$, $T \rightarrow F$, $E \rightarrow T$,
hence count = 3.

For the node corresponding to $a * b$,

a undergoes $F \rightarrow id$ and $T \rightarrow F$ giving 2 reductions,
 b undergoes $F \rightarrow id$ giving 1 reduction,
 then $T \rightarrow T * F$ adds 1 reduction
 and $E \rightarrow T$ adds 1 reduction,
 hence count = $2 + 1 + 1 + 1 = 5$.